A Project Report On

"Advanced Ultrasonic Proximity Detection and Reverse Parking Assistance System"

SUBMITTED TO

Vector India Institute

By

Mr.Benakesh SN(V24BE8B7)
Mr.Darshan Gowda T(V24BE8D6)
Mr.Shreecharan Hebbar(V24BE8S8)
Mr.Shaik Mahammad Hussain(V24BE8S6)

INDEX

Sr. No.	Title	Page No.
1	Abstract	3
2	Introduction	4
3	Objective	5
4	Hardware Requirement	6
5	Software Requirement	6
6	Block Diagram	7
7	Working	7-8
8	Flow of Information	9
9	Therotical Data of Components	10-12
10	Software Component	12
11	Conclusion	13
12	Future Enhancement	13
13	Result	13
14	Code Snippet	14-31

Abstract

This project aims to implement a reverse car parking assistance system using two LPC2129 microcontroller boards (NodeA and NodeB) communicating via CAN protocol. NodeA detects the reverse gear via a push-button and requests distance data from NodeB, which is interfaced with an ultrasonic sensor. Based on the received distance, NodeA controls the buzzer's beep frequency to assist the driver in parking safely.

Introduction

The increasing number of vehicles on roads has made parking assistance systems a necessity. Manual judgment during reverse driving often leads to accidents and damage. Assistance System focus on safety and comfort by reducing human error.

One of the most reliable technologies used in automotive applications is the ultrasonic sensor, which operates on the principle of sound wave reflection. By transmitting ultrasonic waves and measuring their echo time, the system calculates the distance to an obstacle.

In this project, we integrate the ultrasonic sensor with an LPC2129 ARM7 microcontroller and establish communication using the CAN bus. CAN is widely used in automobiles for real-time communication between multiple Electronic Control Units (ECUs). This makes our design scalable, robust, and suitable for realworld automotive applications.

:

Objectives

The objectives of this project are:

- To design and implement an ultrasonic sensor-based obstacle detection system for reverse parking.
- To measure the distance of nearby obstacles using ultrasonic signals.
- To transmit sensor data from the transmitter node to the receiver node using the CAN bus protocol.
- To display the received data or give warnings through UART communication.
- To enhance vehicle safety and reduce accidents during reverse parking.

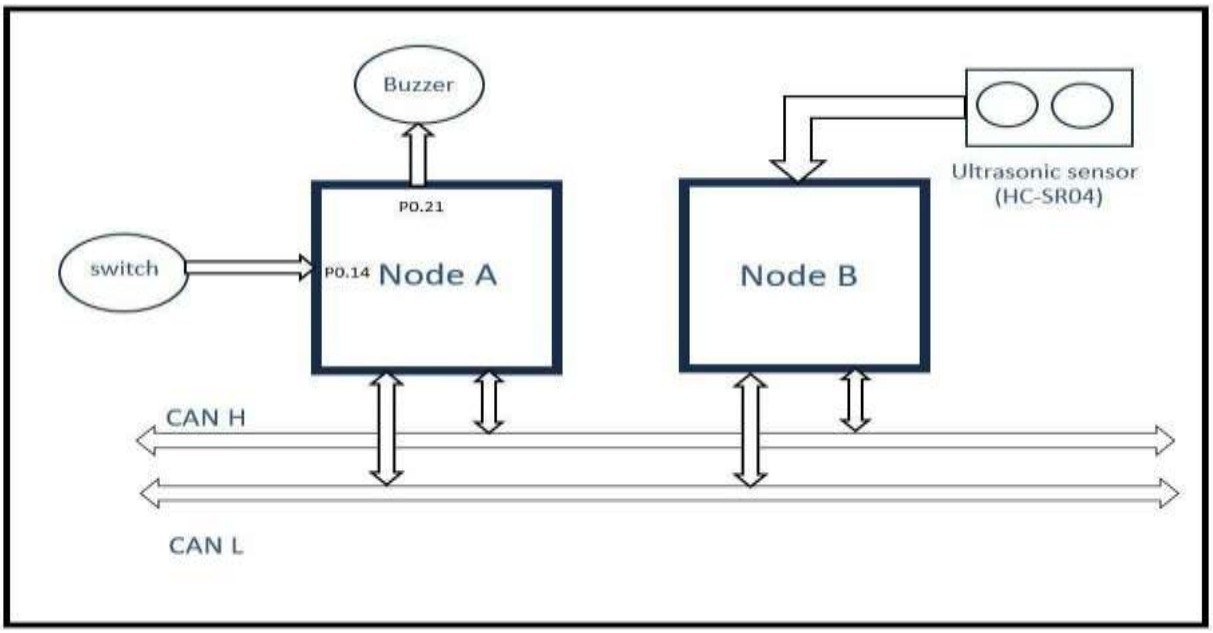
Hardware Requirements

Components Required	Quantity
LPC2129 Boards	2
Ultrasonic Sensor	1
Buzzer	1
Push-button	1
MCP2551 CAN Transceivers	2
120Ω CAN Bus Resistor	1
Power Supply	1

Software Requirements

- Keil µVision (for LPC2129 firmware development).
- Flash Magic (for programming LPC2129).
- Embedded C Language .
- CAN Driver for LPC2129 .
- Delay and Timer modules .
- Ultrasonic Sensor driver code.

Block Diagram



Working:

1) Activation of System (Reverse Gear Detection):

When the driver engages the reverse gear, the push button connected to Node A is pressed.

This acts as the trigger to activate the parking assistance system.

2) Request for Distance Measurement

Once the system is activated, Node A sends a CAN Remote Frame (message ID 0x101) to Node B through the CAN bus.

This frame requests Node B to measure the distance of any obstacle behind the vehicle.

3) **Ultrasonic Distance Measurement (Node B):**

Node B receives the remote frame and triggers the ultrasonic sensor.

The sensor emits a 40 kHz ultrasonic pulse.

The pulse reflects back after hitting an obstacle.

Node B measures the echo time and calculates the distance using the formula:

Distance(cm)= (0.0343*time)/2

The calculated distance value is stored in Node B's register.

4) **Data Transmission via CAN:**

Node B sends a CAN Data Frame (message ID 0x102) containing the distance value (in cm) back to Node A.

This process repeats approximately every 100 ms, ensuring real-time updates.

5) **Alert Generation (Node A):**

Node A receives the distance value and drives the buzzer according to preset distance ranges:

40–30 cm → Buzzer beeps every 500 ms.

29–20 cm → Buzzer beeps every 250 ms.

19–10 cm → Buzzer beeps every 100 ms.

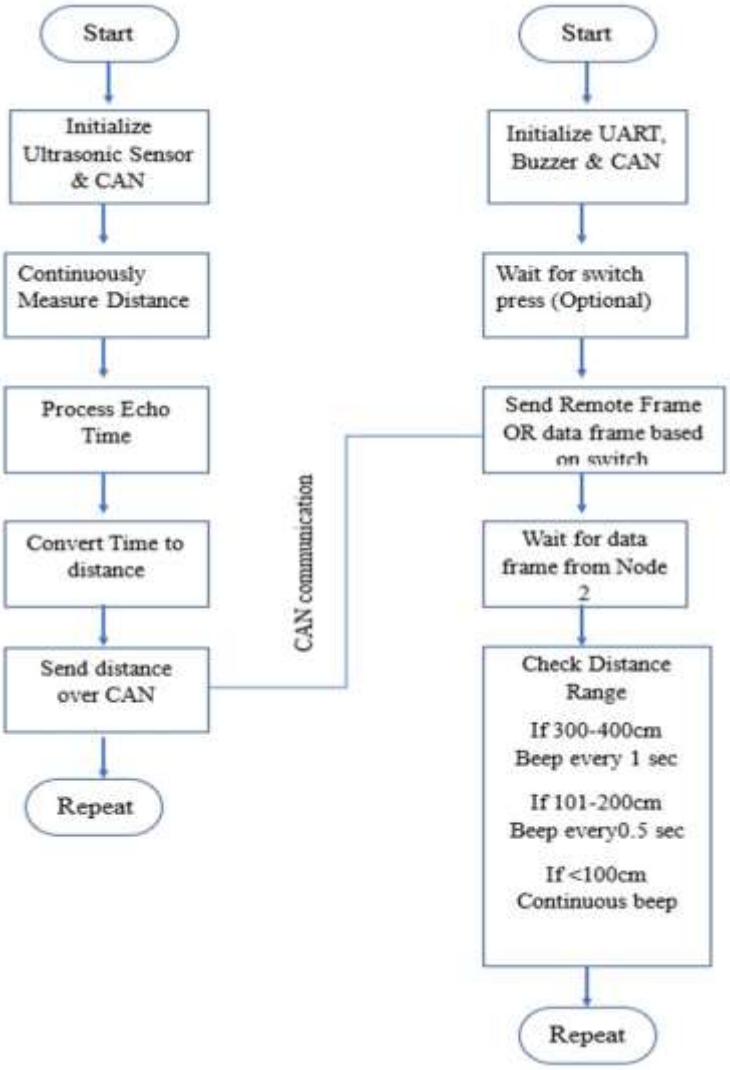
< 10 cm → Continuous buzzer sound (critical warning).

6) **Driver Assistance:**

The driver can judge how close the obstacle is based on the buzzer frequency.

The faster the beeping, the nearer the obstacle. A continuous beep means immediate stopping is required.

Flow Of Information



1. Reverse button pressed → Node A activated.
2. Node A sends CAN request.
3. Node B triggers ultrasonic sensor → measures distance.
4. Node B sends distance value via CAN.
5. Node A receives distance → controls buzzer sound.
6. Buzzer alerts driver with increasing intensity as obstacle nears.

Theoretical Data of Components:

LPC2129 Microcontroller:

- Type: ARM7TDMI-S based 32-bit RISC microcontroller.
- Operating Frequency: Up to 60 MHz.
- Memory: 128 KB on-chip Flash, 16 KB RAM.
- I/O: 46 GPIO pins, multiple timers, UART, SPI, I^2C, and PWM.
- Special Features:
- Built-in two CAN controllers (for Controller Area Network).
- Real-Time Clock (RTC).
- Fast interrupt handling with vectored interrupt controller.
- Why Used: Provides efficient real-time control and has a built-in CAN interface, which makes it suitable for automotive applications like parking assistance.
- MCP2551 CAN Transceiver
- Type: High-speed CAN transceiver by Microchip.
- Operating Voltage: 4.5V – 5.5V.
- Speed: Supports up to 1 Mbps.
- Function:
- Converts the microcontroller's CAN protocol signals (Tx/Rx logic) into differential signals (CAN_H, CAN_L) for the bus.
- Provides noise immunity and error detection.
- Why Used: Ensures reliable communication between Node-A and Node-B through the CAN bus in a noisy automotive environment.

Ultrasonic Sensor (e.g., HC-SR04)

- Operating Principle: Works on the time-of-flight principle of sound waves.
- Emits ultrasonic pulses (40 kHz).
- Measures the time taken for the echo to return.
- Distance = (Speed of Sound × Time) / 2.
- Range: 2 cm – 400 cm.
- Accuracy: ±3 mm.
- Pins:
- TRIG (trigger pulse input).
- ECHO (pulse output).

- VCC, GND.
- Why Used: Provides non-contact distance measurement, making it safe and accurate for obstacle detection.

Buzzer:

- Types: Active (in-built oscillator) or Passive (requires external driving signal).
- Operating Voltage: 3V – 12V.
- Principle: Works on the piezoelectric effect or electromagnetic coil vibration.
- In Project: Used as an alarm/alert device whose beeping frequency increases as the obstacle gets closer.
- Why Used: Simple, cost-effective way to provide immediate feedback to the driver.

Push Button Switch:

- Working Principle: Completes or breaks the circuit when pressed.
- Debouncing: Mechanical contacts can bounce; needs hardware (RC filter) or software debounce.
- In Project: Acts as a reverse gear detection input.
- Why Used: Simulates the reverse gear switch in vehicles, triggering the system when pressed.

CAN Bus (Controller Area Network):

- Type: Multi-master serial communication bus standard.
- Features:
- Two-wire differential signaling (CAN_H, CAN_L).
- Speeds up to 1 Mbps.
- Arbitration using message identifiers (priority-based).
- Robust error handling (CRC, error frames).
- Why Used: Automotive-grade, reliable, long-distance communication between multiple nodes. Perfect for real-time sensor-to-controller communication.

Power Supply:

- Input: Typically 12V (car battery).
- Regulation: Step-down regulators (7805 for 5V, AMS1117-3.3 for 3.3V).
- Why Needed:

- LPC2129 operates at 3.3V.
- MCP2551 & sensors may require 5V.
- Stable, noise-free supply is critical for microcontrollers and CAN bus operation.

Software Components:

Keil µVision IDE:

- Type: Integrated Development Environment (IDE).
- Features:
- Source code editor, compiler, linker.
- Debugger with simulation support.
- Why Used: Supports ARM7 architecture (LPC2129). Provides easy development
- and debugging for embedded C.

Flash Magic:

- Type: ISP (In-System Programming) tool.
- Function: Loads compiled hex file into LPC2129 flash memory via UART.
- Why Used: Reliable and widely used tool for NXP microcontrollers.

Embedded C Language:

- Type: High-level programming language adapted for microcontrollers.
- Features: Portability, direct hardware access, modular design.
- Why Used: Efficient, readable, and suitable for real-time embedded systems.

Result:

The Reverse Car Parking Assistance System was successfully designed, implemented, and tested in real-world conditions. The system provided accurate and real-time feedback to the driver regarding the distance of obstacles behind the vehicle during the reverse motion

Conclusion:

This project demonstrates a real-time application of embedded systems and CAN communication for vehicle safety enhancement.

By utilizing two LPC2129 boards with an ultrasonic sensor and buzzer control logic, the system provides auditory alerts to drivers, helping avoid rear collisions while parking.

Future Enhancements:

1) Add display unit for distance visualization
2) Integrate multiple ultrasonic sensors for wider coverage
3) Implement wireless communication (e.g., Bluetooth) for data log

Code Snippet:

Can1_nodeA_header:-

```
extern void uart0_init(unsigned int);
extern void uart0_tx_string(char *);
extern void uart0_tx_integer(int);
extern void delay_ms(unsigned int);

typedef struct can1_msg {
unsigned int id;
unsigned int  rtr;
unsigned int dlc;
unsigned int byteA;
unsigned int byteB;
}can1;

extern void can1_init(void);
extern void can1_tx(can1);
extern void en_can1_interupt(void);
extern void config_vic_for_can1(void);
```

can1_nodeA_uart_driver:-

```
#include<lpc21xx.h>
#include"header.h"

void uart0_init(unsigned int baud)
{
char a[]={15,60,30,15,15};
unsigned int pclk=0,result=0;
```

```
pclk=a[VPBDIV%4]*1000000;
result=pclk/(16*baud);
PINSEL0 |=0X5;
U0LCR=0X83;
U0DLL= result &0xff;
U0DLM= (result>>8)&0xff;
U0LCR=0X03;
}

#define THRE ((U0LSR>>5)&1)
void uart0_tx(unsigned char data)
{
U0THR=data;
while(THRE==0);
}

void uart0_tx_string(char  *ptr)
{
while(*ptr) {
U0THR=*ptr;
while(THRE==0);
ptr++;
}
}

void uart0_tx_integer(int num)
{
int a[10],i=0;
```

```
if(num==0)
uart0_tx(num+48);
if(num<0)
{
uart0_tx('-');
num=-num;
}
while(num)
{
a[i++]=(num%10)+48;
num=num/10;
}
for(i=i-1; i>=0; i--)
uart0_tx(a[i]);
}

void delay_ms(unsigned int  ms)
{
int a[] ={15,60,30,15,15};
unsigned int pclk;;
pclk=a[VPBDIV]*1000;
T0PC=0;
T0PR=pclk-1;
T0TC=0;
T0TCR=1;
while(T0TC<ms);
T0TCR=0;
}
```

Can1_nodeA_driver:-

```
#include<lpc21xx.h>
#include"header.h"
extern unsigned int flag;
extern can1 r;

void can1_rx_handler(void) __irq
{
r.id=C2RID;
r.dlc=(C2RFS>>16)&0XF;
r.rtr=(C2RFS>>30)&1;
if(r.rtr==0)
{
r.byteA=C2RDA;
r.byteB=C2RDB;
}
C2CMR=(1<<2);
flag=1;
VICVectAddr=0;
}

void en_can1_interupt(void)
{
C2IER=1;
}
```

```
void config_vic_for_can1(void)
{
VICIntSelect=0;
VICVectCntl2=27|(1<<5);
VICVectAddr2=(unsigned int) can1_rx_handler;
VICIntEnable=(1<<27);
}

void can1_init()
{
VPBDIV=1;
PINSEL1|=0X14000;
C2MOD=1;
AFMR=2;
C2BTR=0X001C001D;
C2MOD=0;
}

#define TCS (C2GSR&8)
void can1_tx(can1 v)
{
C2TID1=v.id;
C2TFI1=(v.dlc<<16);
if(v.rtr==0)
{
C2TDA1=v.byteA;
C2TDB1=v.byteB;
}
```

```
else
C2TFI1|=(1<<30);
C2CMR=1|(1<<5);
while(TCS==0);
}
```

Can1_nodeA_main:-

```
#include<lpc21xx.h>
#include"header.h"
#define RG      ((IOPIN0>>14)&1)
#define LED     (1<<17)
#define BUZZER  (1<<21)
#define LED1    (1<<18)

can1 v1,v2,r;
unsigned int flag;
int main()
{
unsigned int sw=0,data_flag=0;
IODIR0=RG|LED|BUZZER|LED1;
IOSET0=LED;
can1_init();
uart0_init(9600);
en_can1_interupt();
config_vic_for_can1();

v1.id=0x123;
v1.dlc=2;
```

```
while(1)
    {
  if(RG==0)
      {
   while(RG==0);
    sw^=1;
        if(sw)
          {
   IOCLR0=LED;
uart0_tx_string("Reverse gear is enabled\r\n");
    data_flag=0;
      v1.rtr=1;
    can1_tx(v1);
          }
        else
          {
   IOSET0=LED;
  IOCLR0=LED1;
IOCLR0=BUZZER;
    data_flag=1;
      v1.rtr=0;
    can1_tx(v1);
uart0_tx_string("Reverse gear is disabled\r\n");
          }
      }

 if(data_flag)
```

```
IOCLR0=LED1;

if(flag==1)
{
flag=0;
v2.id=r.id;
v2.dlc=r.dlc;
if(v2.rtr==0)
{
v2.byteA=r.byteA;
uart0_tx_integer(v2.byteA);
uart0_tx_string("cm\r\n");
}

if(v2.byteA>=30 && v2.byteA<=40)
{
IOSET0=LED1;
IOSET0=BUZZER;
delay_ms(500);
IOCLR0=LED1;
IOCLR0=BUZZER;
delay_ms(500);
}

else if(v2.byteA>=20 && v2.byteA<=29)
{
IOSET0=LED1;
IOSET0=BUZZER;
```

```
delay_ms(250);
IOCLR0=LED1;
IOCLR0=BUZZER;
delay_ms(250);
}

else if(v2.byteA>=10  && v2.byteA<=19)
{
IOSET0=LED1;
IOSET0=BUZZER;
delay_ms(100);
IOCLR0=LED1;
IOCLR0=BUZZER;
delay_ms(100);
}

else if(v2.byteA>=1 && v2.byteA<=9)
{
IOCLR0=LED1;
IOSET0=BUZZER;
}
}
}
}
```

Can1_nodeB_header:-

```
extern void uart0_init(unsigned int);
extern void uart0_tx_string(char *);
```

```
extern void uart0_tx_integer(int);
extern void delay_ms(unsigned int);
extern void delay_us(unsigned int);

typedef struct can1_msg {

unsigned int id;
unsigned int rtr;
unsigned int dlc;
unsigned int byteA;
unsigned int byteB;

}can1;

extern void can1_init(void);
extern void can1_tx(can1);
extern void en_can1_interupt(void);
extern void config_vic_for_can1(void);

extern void ultrasonic_init(void);
extern void send_pulse(void);
extern unsigned int get_range(void);
```

can1_nodeB_uart_driver:-

```
#include<lpc21xx.h>
#include"header.h"

void uart0_init(unsigned int baud)
```

```
{
char a[]={15,60,30,15,15};
unsigned int pclk=0,result=0;
pclk=a[VPBDIV%4]*1000000;
result=pclk/(16*baud);
PINSEL0 |=0X5;
U0LCR=0X83;
U0DLL=result &0xff;
U0DLM=(result>>8)&0xff;
U0LCR=0X03;
}

#define THRE ((U0LSR>>5)&1)
void uart0_tx(unsigned char data)
{
U0THR=data;
while(THRE==0);
}

void uart0_tx_string(char  *ptr)
{
while(*ptr) {
U0THR=*ptr;
while(THRE==0);
ptr++;
}
}
```

```
void uart0_tx_integer(int num)
{
int a[10],i=0;
if(num==0)
uart0_tx(num+48);
if(num<0)
{
uart0_tx('-');
num=-num;
}
while(num)
{
a[i++]=(num%10)+48;
num=num/10;
}
for(i=i-1; i>=0; i--)
uart0_tx(a[i]);
}

void delay_ms(unsigned int  ms)
{
int a[] ={15,60,30,15,15};
unsigned int pclk;;
pclk=a[VPBDIV]*1000;
T0PC=0;
T0PR=pclk-1;
T0TC=0;
T0TCR=1;
```

```
while(T0TC<ms);
T0TCR=0;
}

void delay_us(unsigned int us)
{
T1PC=0;
T1TC=0;
T1PR=59;
T1TCR=0X02;
T1TCR=1;
while(T1TC<us);
T1TCR=0;
}
```

Can1_nodeB_driver:-

```
#include<lpc21xx.h>
#include"header.h"
extern unsigned int flag;
extern can1 r;

void can1_rx_handler(void) __irq
{
r.id=C2RID;
r.dlc=(C2RFS>>16)&0XF;
r.rtr=(C2RFS>>30)&1;
if(r.rtr==0)
{
```

```
r.byteA=C2RDA;
r.byteB=C2RDB;
}
C2CMR=(1<<2);
flag=1;
VICVectAddr=0;
}

void en_can1_interupt(void)
{
C2IER=1;
}


void config_vic_for_can1(void)
{
VICIntSelect=0;
VICVectCntl2=27|(1<<5);
VICVectAddr2=(unsigned int) can1_rx_handler;
VICIntEnable=(1<<27);
}

void can1_init()
{
VPBDIV=1;
PINSEL1|=0X14000;
C2MOD=1;
AFMR=2;
```

```
C2BTR=0X001C001D;
C2MOD=0;
}

#define TCS (C2GSR&8)
void can1_tx(can1 v)
{
C2TID1=v.id;
C2TFI1=(v.dlc<<16);
if(v.rtr==0)
{
C2TDA1=v.byteA;
C2TDB1=v.byteB;
}
else
C2TFI1|=(1<<30);
C2CMR=1|(1<<5);
while(TCS==0);
}
```

Can1_nodeB_ultrasonic_driver:-

```
#include<lpc21xx.h>
#include"header.h"
#define TRIG (1<<2)
#define ECHO (1<<3)
```

```
void ultrasonic_init(void)
{
IODIR0|=TRIG;
IODIR0&=~(ECHO);
T0TCR=0x02;
T0PR=59;
T0TCR=1;
}

void send_pulse(void)
{
T0TC=0;
T0PC=0;
T0PR=59;
IOSET0=TRIG;
delay_us(10);
IOCLR0=TRIG;
IODIR0&=~(ECHO);
}

unsigned int get_range(void)
{
unsigned int time,distance;
send_pulse();
while(!((IO0PIN&ECHO)));
T0TCR=1;
while((IO0PIN&ECHO));
T0TCR=0;
```

```
time=T0TC;
distance=(0.0343*time)/2;
return distance;
}
```

Can1_nodeB_main:-

```
#include <lpc21xx.h>
#include "header.h"
can1 r1,r2,r;
unsigned int flag;

int main()
{
unsigned int distance,r;
can1_init();
config_vic_for_can1();
en_can1_interupt();
ultrasonic_init();
uart0_init(9600);

while(1)
{
if(flag==1)
{
flag=0;
if(r1.rtr==1)
{
r=1;
```

```
r2.dlc=r1.dlc;
r2.id=r1.id;
}
else
r=0;
}
if(r==1)
{
distance=get_range();
delay_ms(50);
uart0_tx_integer(distance);
r2.byteA=distance;
r2.rtr=0;
can1_tx(r2);
}
}
```